

Lab 15 AI security evaluation

AI for IT Security Professionals | TGS-2023039344 | v6.0

Purpose and output

Create a confusion matrix and holdout evaluation plan. This lab supports LO1. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: `REVIEW_ERRORS`. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

3 Trace the mechanism

AI evaluation holdout

Use separate tuning and holdout examples to estimate performance on unseen cases.

Contract: `dataset_split`, `label`, `prediction`, `threshold`

Accepted case: The threshold is chosen on tuning data and frozen for holdout.

Counterexample: The holdout labels are used to optimise the same threshold.

Diagnosis: Create a fresh holdout and document the data boundary; estimates from contaminated data are optimistic.

Evidence: Retain split membership, fixed threshold and error analysis.

Adversarial input resilience

Evaluate both benign and malicious context against the same external control boundary.

Contract: case_id, attack_type, expected_action, observed_action

Accepted case: A malicious instruction still produces a denied privileged action.

Counterexample: A harmless-looking paraphrase bypasses a keyword-only filter.

Diagnosis: Broaden the test set and enforce authorisation independent of keyword detection.

Evidence: Report bypasses and false alarms with the exact test-case IDs.

Model and dependency provenance

An approved release binds model version, prompt template and dependency checksums.

Contract: model_version, prompt_hash, package_hash, approval

Accepted case: All artefacts match the approved release manifest.

Counterexample: An unreviewed package changes tool behaviour.

Diagnosis: Block rollout and restore the approved artifact set; signature and origin checks complement scanning.

Evidence: Keep the manifest and a verified reproducibility record.

4 Change one input and test the consequence

Save a copy of mock-data.json as changed-data.json using the editor. Change case-3 predicted to true. Recall becomes 1.0 while precision becomes 0.75. Explain why this single fixture edit is not proof of an improved detector on unseen data.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

5 Review AI claims against evidence

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction.

Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

6 Verify the corresponding operational control

Use the labelled synthetic cases only. Record model/prompt version if an approved model is evaluated. Keep the labels out of the prediction prompt, freeze the prompt before the holdout run, and compare case IDs. Do not label the deterministic fixture runner as a trained AI detector.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a confusion matrix and holdout evaluation plan with an owner, evidence and remaining uncertainty.

Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with `python3 -m json.tool changed-data.json`; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.
- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.